

Qblox Information

Kyle MacRobbie

Last updated: 10 December 2024

Contents

1	Introduction	3
1.1	Purpose of this document	3
1.2	The Cluster	3
2	Modules	3
2.1	Specifications	3
2.2	Outputting when nothing should be playing	4
2.3	Output linearity	4
2.4	Scaled inputs	6
3	Code	6
3.1	Acquisition lengths	6
3.2	Selecting a specific input or output	6
3.3	Enabling modulation	7
3.4	Mixer calibration	8
3.5	Averaging	8
3.6	Phase control in Python	8
4	Q1ASM	8
4.1	Playing waveforms	10
4.2	Acquisitions	11
4.2.1	acquire	11
4.2.2	acquire_weighed	11
4.2.3	acquire_ttl	12
4.3	Acquisitions longer than 16 μ s	12
4.4	Registers	13
4.5	Looping	13
4.6	Phase control in Q1ASM	14
4.7	Other useful commands	14
4.8	sequence_helper library	15
5	Use With Other Equipment	15
5.1	Oscilloscope	15
5.2	Frequency mixer	16
5.3	Lock-in amplifier	17

6	Updating Firmware and Drivers	17
6.1	Updating qblox-instruments	17
6.2	Updating firmware	18
7	SPI Rack	19
7.1	Specifications	19
7.2	Use with Python	19
7.3	Use with Spanish Acquisition	20
8	Slack Channel	21

1 Introduction

1.1 Purpose of this document

This document is meant to speed up the process of learning how to use the Qblox cluster, and clarify the confusions that we had when trying to learn how to use it, so that you don't have to go through the process of figuring it out by yourself. If you run into confusion when learning how to use Qblox, refer to this document, your question might just be answered here.

If you have something that you think should be changed/added to this document, the TeX file can be found in the CSG Teams files.

1.2 The Cluster

The Cluster is simply the big blue box on the shelf that says QBLOX on the side and houses all the modules. The Cluster can be turned on using the switch found at the back. Once turned on, the LEDs on the modules will appear yellow for a few moments before going green to indicate that it is on and ready for use.

Any input or output not in use should be terminated with a $50\ \Omega$ SMA-M termination.

2 Modules

There are two baseband modules, the qubit control module (QCM) and the qubit readout module (QRM). We have one radio frequency module, the qubit control module - radio frequency (QCM-RF).

2.1 Specifications

Parameter	QCM	QRM	QCM-RF
# of outputs	4	2	2
# of inputs	0	2	0
# of marker ports	4	4	2
I/O connection	SMA-F	SMA-F	SMA-F
Marker connection	SMP-M (pin)	SMP-M (pin)	SMP-M (pin)
Output voltage (V)	± 2.5	± 0.5	± 0.5
Input voltage (V)	-	± 1.0	-
I/O impedance (Ω)	50	50	50
Frequency range	± 500 MHz	± 500 MHz	2-18 GHz

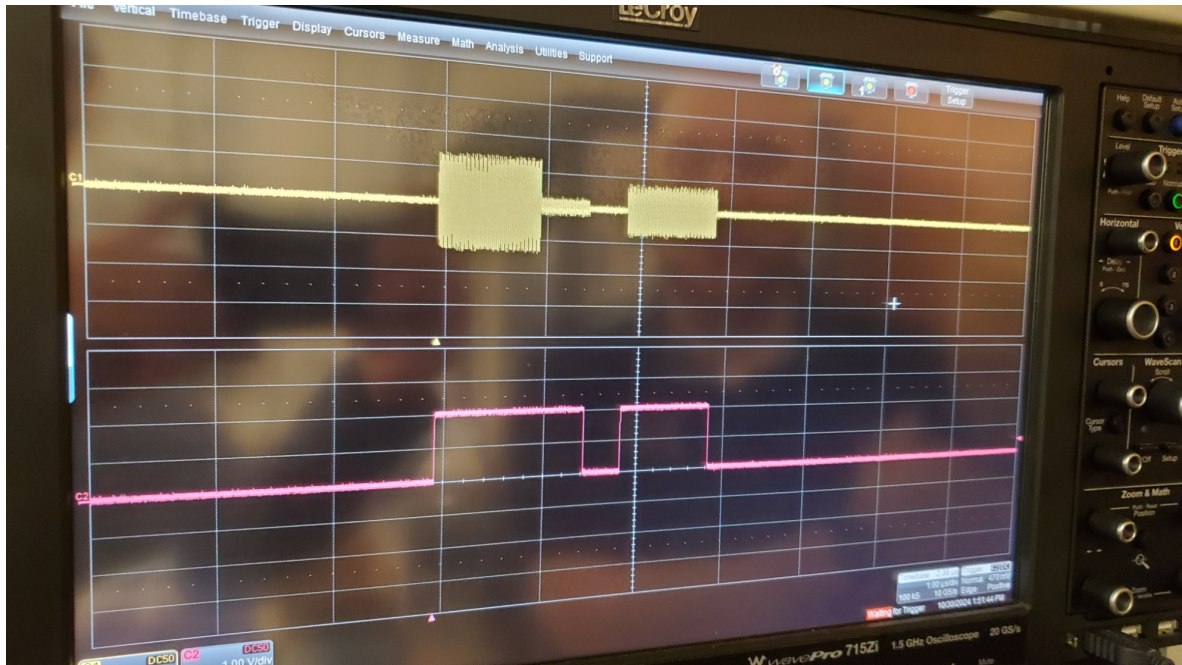
At first glance, negative frequency might not make any sense. The explanation for this is that the waves are in the complex form $e^{i\omega t}$, so ω could be positive or negative, and this would affect the polarity of the wave on the complex path.

Make is that when viewing signals with the oscilloscope, make sure that the input coupling on the oscilloscope matches the impedance of the module output, or else your results on the oscilloscope will be scales by some factor and appear incorrect.

An important note is that in order to either modulate the NCO in any of the modules, or to use the LO in the QCM-RF, you *must* manually enable modulation. Instructions for this is in the code section of this document.

2.2 Outputting when nothing should be playing

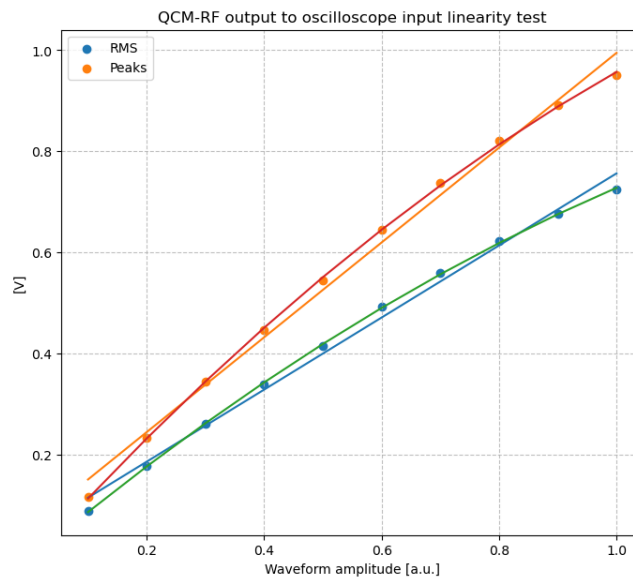
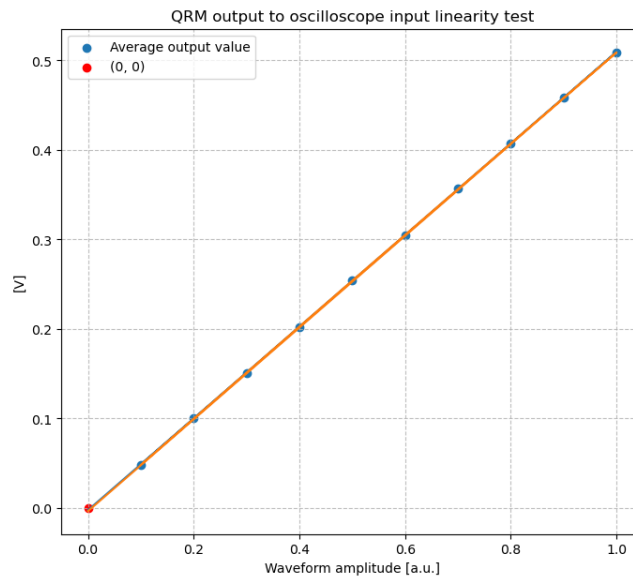
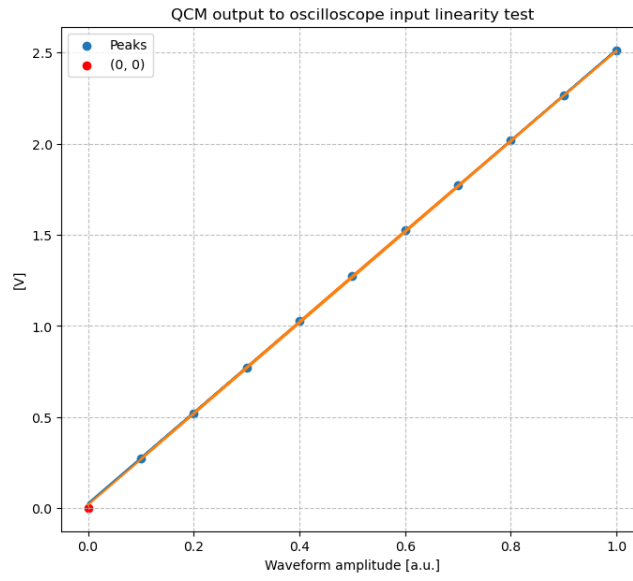
When we were working with the QCM-RF, we noticed that if a play command was longer than the duration of the waveform we were playing, we would still get some output after the waveform was done, that was greater than the background noise.



The cause of this is likely LO leakage through the modules internal mixer. To remedy this, you can better calibrate the mixer to try to get rid of as much leakage as you can, or do not have the play commands run longer than the waveforms.

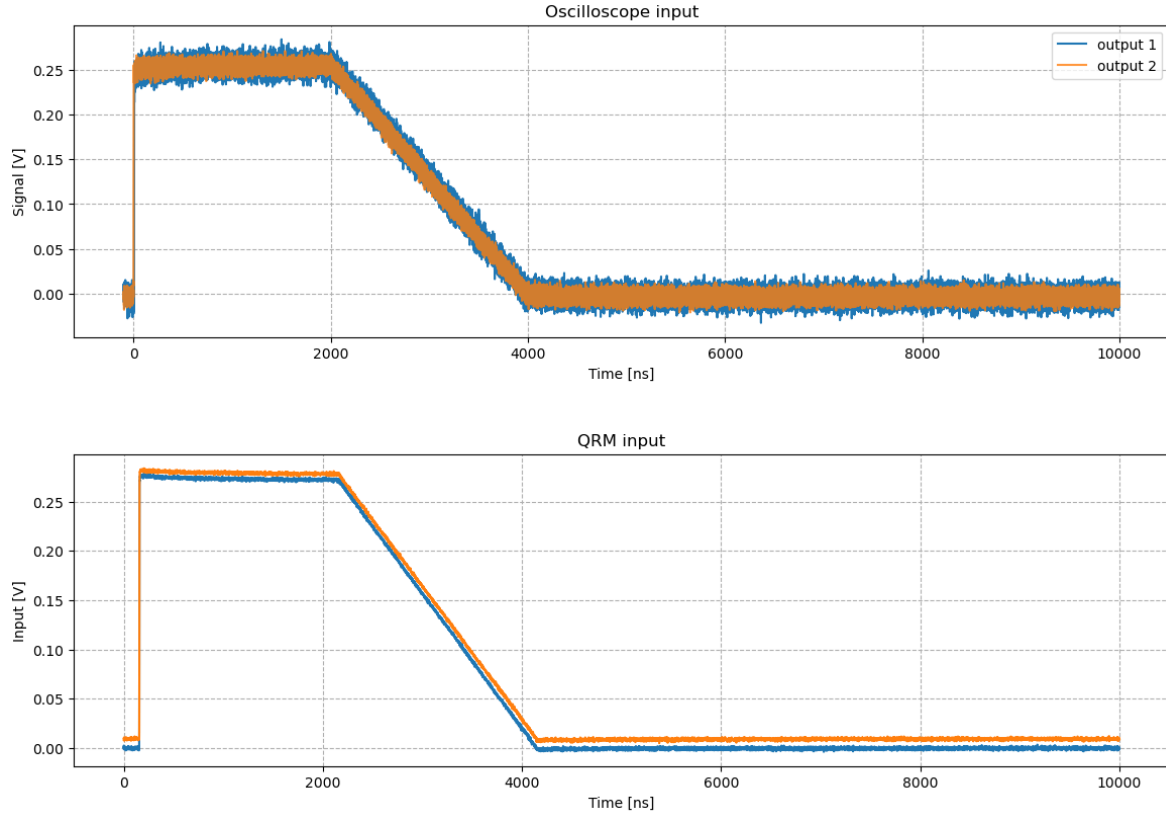
2.3 Output linearity

Some Qblox outputs will not behave in exactly the way you would expect. For example, if you output a waveform defined in a Python dictionary at an amplitude of 1, then the same waveform at an amplitude of 0.5, you would expect that the resulting output from the module you are using would be exactly half the size however, with the QCM-RF module, this is not always the case. The following are plots for the outputs of each of the modules oscilloscope. You can see that the baseband modules are much more linear than the RF modules.



2.4 Scaled inputs

The QRM seems to consistently scale its inputs to be 10% greater than what it should actually be. The following plots shows the same sequence being outputted to an oscilloscope and the QRM. The pulse is programmed to be at 250 mV, and the oscilloscope reads this correctly, while the QRM reads around 10% greater.



3 Code

3.1 Acquisition lengths

The maximum acquisition time allowed is 16384 ns. Acquisitions will run for the entire 16384 ns unless they are stopped. If another acquisition is met, the current one will be stopped. But if you are integrating and binning, you don't have to integrate over the entire range, you can set the number of points that it integrates over using:

```
module.sequencer.integration_length_acq(1234)
```

With the number of points used being the argument to this function.

3.2 Selecting a specific input or output

In order to output on a specific output, you can use the following line of code:

```
module.sequencer.connect_out0(True)
```

To connect to a different output, simply change the number after "out" to match the index of the output you wish to use. Note that the index number and the output number on the front of the physical module are not the same. Output 1 on the module is index 0, output 2 is index 1, etc. The number used in the code is the index.

To connect to a specific input, you may have to change more than one line of code, depending on the type of acquisition you are doing. Again, note that you may have to change the index numbers depending on the input you are using.

- `module.in1_gain(input_gain)`: To set the input gain on input index 1.
- `module.sequencer1.ttl_acq_threshold(threshold)`: to change TTL threshold on input index 1.
- `module.sequencer1.connect_acq_I("in1")`: Selecting input with index 1.
- `module.ttl_acq_input_select(1)`: When using TTL acquisition.

3.3 Enabling modulation

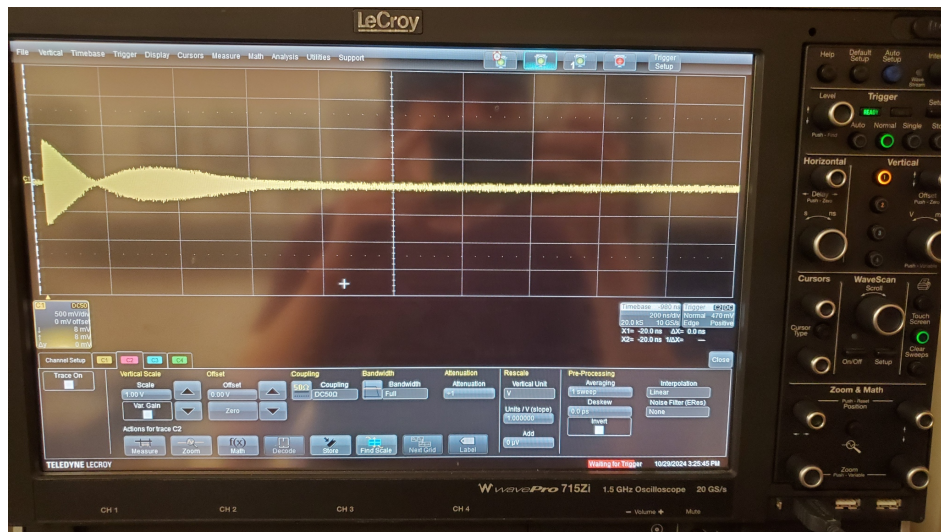
You need to manually enable the modulation of the NCO, or else it will output as if the NCO is set to 0 Hz, which will give very strange outputs contrary to what you would expect. Enable modulation on each sequencer that you need using the following (using sequencer 0, for example):

```
module.sequencer0.mod_en_awg(True)
```

You must also do the same for the LO.

```
module.out0_lo_en(True)
```

If the modulation of the NCO is not enabled or is set too low, it will appear as the following when trying to play a simple square pulse.



From a discussion we had with a Qblox representative, less than 50-100 MHz should not be necessary.

3.4 Mixer calibration

Automatic mixer calibration can be done on the LO leakage and undesired sideband by using the following lines (on output 0 and sequencer 0, for example).

```
module.out0_lo_cal()  
module.sequencer0.sideband_cal()
```

Note that the LO and NCO frequency need to be set before running these functions in order for the calibration to work properly.

Manual mixer calibration can also be used, and will likely result in a better calibration, however, you need a spectrum analyzer. See the *Manual Mixer Calibration* tutorial on the Qblox website for instructions.

3.5 Averaging

Hardware averaging can be enabled to do many loops in the Q1ASM program and average the results into a single acquisition. By simply using the line of code:

```
module.scope_acq_avg_mode_en_path0(True)
```

The outputted acquisition for path 0 in this case will be all the acquisitions looped through in Q1ASM averaged with each other, reducing noise and giving a much clearer output.

3.6 Phase control in Python

The phase of the RF signal can be controlled using the Python command:

```
sequencer.nco_phase_offs()
```

Where the argument is the phase in degrees. Note that this is much slower than controlling the phase in Q1ASM.

4 Q1ASM

Q1ASM is an assembly language made by Qblox in order to run all sequences performed by Qblox modules. The Q1ASM sequence itself is stored in a string in Python to be passed to a sequencer within the modules. The sequence string is then passed to a sequence dictionary under the label *program*, which is then passed to the sequencer using the line (using sequencer 0, for example):

```
module.sequencer0.sequence(sequence_dictionary)
```

The following is an example of a Q1ASM being defined and passed to a sequencer.


```

seq_mark = f"""
    move      100000000,R0

    wait_sync 4
    set_mrk   {0b1001}
    upd_param 4

    set_freq  R0
    play      0,0,1000
    wait      1000

    add       R0,300000000,R0

    set_freq  R0
    play      0,0,1000

    set_mrk   {0b0000}
    upd_param 4
    stop
"""
sequence_mark = {
    "waveforms": waveforms,
    "weights": {},
    "acquisitions": {},
    "program": seq_mark,
}
rf_module.sequencer0.sequence(sequence_mark)

```

The commands in Q1ASM contain an *instruction* and a *signature*. The instruction is simply the same of the command, and the signature is the value(s) that are being fed into the command, like arguments in a function. For example, in the command:

```
play    0,0,5000
```

play is the instruction and *0*, *0* and *5000* are the signatures. Note that Q1ASM uses white space to separate the command from the signatures but uses commas to separate the signatures from each other.

Comments can be added anywhere in the Q1ASM sequence, which is denoted by a *#*, proceeded by the comment, for example:

```
play    0,0,5000
# This is a comment
```

Any command can also have a label that serves as a sort of address for that command. This is accomplished by writing the command name followed by a *:*, all before the instruction. Labels can be given any name, and will be called using whatever name you give it. For example, we could name our label *loop*, and define it as follows:

```
loop:   play    0,0,5000    # Beginning of my loop
```

As just foreshadowed, labels are essential to looping, which will be elaborated on later in this document.

A useful tool when making Q1ASM programs is the f-string in Python. By placing an *f* before of a string, you can insert variables previously defined in Python in curly brackets, making changing your sequence easier if it needs to be done later down the line. Shown below is an example.

```
# Readout sequence
seq_readout = f"""
    move      {num_iterations},R0      # Loop iterator

loop:
    wait_sync 4
    wait      {gate_pulse_length*3}  # Wait until step 4

    acquire   0,0,{rf_pulse_length}

    wait      {gate_pulse_length*2}
    loop      R0,@loop
    stop
"""
```

4.1 Playing waveforms

To play a waveform, you must first define the waveform outside of Q1ASM as follows:

```
waveform_length = 1000 # nanoseconds
waveforms = {
    "block": {
        "data": [1.0 for i in range(0, waveform_length)],
        "index": 0,
    }
}
```

The *waveforms* dictionary contains all defined waveforms. Here, *block* is the name of a waveform, which itself is another dictionary containing a list labeled *data* and an integer labeled *index*. *data* is just a list containing all the the data points of you waveform, with each point being 1 ns apart. *label* is an integer that you assign the waveform which will be used to call that waveform to be played in the Q1ASM program. It is important to not give two waveforms the same name or index.

Now that you have defined a waveform, you can play it in Q1ASM using the *play* command:

```
play    0,0,2000
```

This command will play the waveform with index 0 on path 0, the waveform with index 0 on path 1, and wait at least 2000 ns before any other real-time command is allowed to be executed. This means that if we include another *play* command after,

```
play    0,0,2000
play    0,0,1000
```

Since our waveform with index 0 is only 1000 ns long, the waveform will play for the first 1000 ns, the nothing will be played as the first *play* command waits out the remaining 1000 ns of its time, before the second *play* command can start playing its waveform.

4.2 Acquisitions

There are 3 different types of acquisitions that can be performed with the QRM, a regular acquisition, a weighed acquisition, and a TTL acquisition.

Just like waveforms, acquisition must be defined outside of Q1ASM before they can be called by Q1ASM sequence. This is done as follows:

```
acquisitions = {
    "non_weighed": {"num_bins": 10, "index": 0},
    "weighed": {"num_bins": 10, "index": 1},
    "large": {"num_bins": 16384, "index": 2},
    "avg": {"num_bins": 10, "index": 3},
    "single": {"num_bins": 1, "index": 4},
}
```

Like the waveforms, the *acquisitions* dictionary contains all defined acquisitions. The acquisitions themselves contain two integers, *num_bins* and *index*. *num_bins* is the number of bins this acquisition will have available, and *index* is how the acquisition will be called in the Q1ASM sequence. The QRM supports up to 131072 bins. When an acquisition occurs, it also integrates over all the data and stores the results in a bin. The bin information of all the acquisitions can be retrieved after the acquisitions are done, however one the last run of the acquisition will have its raw data available, as this get overwritten whenever a new *acquire* command is met.

4.2.1 acquire

The *acquire* command simple begins reading, and will continue reading for 16384 ns unless it is stopped sooner. The command is formatted as follows:

```
acquire    2,3,1000
```

This command will perform an acquisition, using the defined acquisition with index 2, store the integration information in the bin with index 3, and wait for at least 1000 ns before the acquisition can be interrupted. If it is interrupted, the command will only retrieve 1000 ns worth of data, but if it is not interrupted, it will continue to its maximum of 16384 ns before finally stopping. The length of the integration can be set as described in the code section of this document.

4.2.2 acquire_weighed

This command works similarly to *acquire*, with the difference being that before it integrates acquisition data, it puts a weight over the acquisition. These weights must also be defined outside of the Q1ASM sequence before they can be called, as such:

```
waveforms_weights = {
    "gaussian": {
        "data": scipy.signal.windows.gaussian(
            waveform_weight_length, std=0.12 * waveform_weight_length
        ).tolist(),
        "index": 0,
    },
}
```

The *waveform_weights* contains one weight name *gaussian* which contains a list *data* and an integer *index* which is used to call the weight from the Q1ASM sequence.

To acquire data and perform this weight on the data before integrating, the following command can be used:

```
acquire_weighed 2,3,0,0,1000
```

This command will perform an acquisition, using the acquisition with index 2, store the weighed integration in the bin with index 3, using the weight with index 0 on path 0, and the weight with index 0 on path 1, and wait for a minimum of 1000 ns before beginning another acquisition.

4.2.3 acquire_ttl

TTL acquisition can keep track of how many pulses have passes some threshold in the acquisition. The threshold is defined using the following line (for sequencer 0, for example):

```
module.sequencer0.ttl_acq_threshold(0.5) # Threshold in volts
```

The information about the number of detected pulses is stored in the *avg_cnt* label of the acquisition dictionary.

4.3 Acquisitions longer than 16 μ s

Acquisitions longer than 16384 ns are possible by using binning, at the expense of the resolution of the acquisition. This is accomplished by looping *acquire* statements with a length of however long you want your resolution to be (say for example, 500 ns) and then integrating that acquisition into a bin, using an integration length of 500. The value in this bin is now the sum of the first 500 ns (500 data points) of your acquisition. By then dividing this value by 500, the value in this bin is now one data point representing the average input value of that 500 ns. Since qblox supports 131072 bins, you can have an acquisition time of your resolution multiplied by the number of bins your are using, giving a maximum of:

$$\text{max acquisition time} = \text{resolution} \times 131072$$

Since the maximum resolution is the maximum length of one acquisition (16384 ns), the maximum possible acquisition with the QRM is :

$$\text{absolute max acquisition time} = 16384 \text{ ns} \times 131072 = 2.15 \text{ s}$$

Qblox can only loop, acquire, integrate and bin so quickly. For this reason, you can't just pick any value to be your resolution. From my testing, the minimum resolution you can have for this method of acquisition is 220 ns. If you need a resolution faster than 220 ns, you will be limited to the 16 μ s of acquisition time.

The following is an example of a Q1ASM program that will collect data for 10 ms:

```

1  seq = f"""
2      move    {num_loops},R0  # Loop iterator
3      move    0,R1           # Bin iterator
4
5      wait_sync 4
6
7      loop:
8          acquire 0,R1,{resolution}
9          add     R1,1,R1
10         loop    R0,@loop
11
12         stop
13 """

```

4.4 Registers

In Q1ASM, registers are used as variables. There are 64 registers available to use for each sequencer and are named by putting an R in front of their number (R0 through R63). Registers become very important when looping in order to increment values and control the number of loops executed. Registers can be defined and assigned a value as follows:

```
move    5,R0
```

This *moves* the value 5 into the register R0. Until its value is changed, any time the register R0 is used, it will have a value of 5.

4.5 Looping

Looping relies on the usage of indexes and registers. The following is an example of a loop in Q1ASM:

```

move    0,R0
loop:   add    R0,1,R0
play    1,1,1000
wait    500
jlt     R0,10,@loop
stop

```

New commands in this sequence are *add*, *wait*, *jlt* and *stop*. *stop* simply stops the sequencer, marking the end of the sequence. The *add* command will take the first two signatures, add them and move that value to the third signature. So in this case, it will add 1 to the value of R0, and move that value to R0, essentially just incrementing R0 by 1. *wait* will pause the sequence for 500 ns before moving on to the next command. The *jlt* command will check to see if R0 is less than 10, and if this is true, it will jump to the place in the sequence with the *loop* label. If the command is false, it will simply continue on to the next command in the sequence.

We have now created a loop, where waveform 1 will play 10 times for 1000 ns each, with 500 ns between each pulse, then the sequence will stop.

Another command that can be used as a logic gate in place of *jlt* is *jge*:

```
jge    R0,10,@loop
```

This will check if R0 is greater than or equal to 10, and if this is true, jump to the *loop* label. Else, it continues to the next command in the sequence.

In addition, as opposed to incrementing values with *add*, you can decrement values with *sub*, using the same syntax.

Another, more simple logic gate is the *loop* command, which only takes a register and label as its signatures, and it will decrement the register value by 1, and go to the label if the value is non-zero. For example:

```
loop   R0,@loop
```

Will decrement R0 by 1 and jump to the *loop* label. To loop *n* times, simply set R0 to be *n* before the loop. This notation can be a bit confusing, using *loop* as both the command and the name of the label, so just note here that the *loop* used in the command and the *loop* in *@loop* are different, one being a command and the other being a label.

4.6 Phase control in Q1ASM

The RF pulse phase can be controlled in Q1ASM with the following command:

```
set_ph
```

This takes a number from 1 to 10^9 , linearly mapping to 0° to 360° . For example, setting the phase to 90° would use the following command:

```
set_ph    {250e6}
```

4.7 Other useful commands

All Q1ASM commands can be found at https://docs.qblox.com/en/main/cluster/q1_sequence_processor.html#q1asm, but here are some particularly useful ones:

- **set_mrk**

This command enables the markers on the modules. The syntax uses a *0b* followed by 4 binary digits (either 0 or 1). On the QCM and QRM, these binary digits correspond to the state of the 4 marker channels. For example, if you wished to turn on the first and third markers, you would use the following command:

```
set_mrk    {0b1010}
```

For the QCM-RF, the first two digits will enable outputs 1 and 2, while the third digit controls the second marker, and the fourth digit control the first marker output, so if you only wanted to use the first output and the first marker, you would use the following:

```
set_mrk    {0b1001}
```

- **upd_param**

This command updates any latched parameters, for example, the marker state. This means the marker state defined in **set_mrk** will not actually be executed until **upd_param** is met. The only signature that **upd_param** takes is the wait time that will be met once the command is ran, which is a minimum of 4. The syntax is as follows (waiting 1000 ns, for example):

```
upd_param    1000
```

- **set_freq**

This command sets the frequency of the NCO. Note that for the QCM-RF, the LO frequency cannot be set using Q1ASM. This command takes an integer, having 4×10^9 steps between -500 MHz and 500 MHz, meaning the value you put into the Q1ASM signature is:

$$signature = 4 \times \omega$$

For example, if you wished to set the NCO frequency to 1 MHz, you would use the following command.

```
set_freq    4000000    # Sets NCO to 1 MHz
```

- **wait_sync 4**

This command can be put at the start of all Q1ASM sequences when using multiple sequencers together in order to make sure that the sequencers are synced with each other before beginning to perform any other commands.

4.8 sequence_helper library

I (Kyle) made a library of helper functions during my co-op in order to make sequencing and connecting to inputs/outputs easier and quicker. See the library and a tutorial on how to use it in Qblox Master Folder > Cluster > Libraries > Qblox Sequence Helper.

5 Use With Other Equipment

5.1 Oscilloscope

The outputs of the modules can be hooked up to the LeCroy WavePro 715Zi oscilloscope located on the shelf right above the Qblox Cluster. You can hook up the markers to the oscilloscope as well, then set the oscilloscope to trigger on the marker. This is much easier than trying to trigger off of the signal itself.

Make sure to set the impedance on the oscilloscope to match the impedance of the output you are using, or else the voltage values on the oscilloscope will be scaled by some value.

Some parameters of the oscilloscope can be controlled in the Jupyter notebook using PyVISA. The following code will connect to the oscilloscope.

```
[102] 1 # Get a list of all connected devices
      2 rm = pyvisa.ResourceManager()
      3 rm.list_resources()
Python

... ('ASRL3::INSTR', 'GPIB0::8::INSTR')

[103] 1 # Connect to the oscilloscope
      2 scope = rm.open_resource('GPIB0::8::INSTR')
      3 print(scope.query('*IDN?'))
Python

... *IDN LECROY,WP715ZI,LCRY0716N47852,8.5.0
```

Once connected, the following commands can be used:

- `scope.write(f'TIME_DIV {15e-6} S')`

This will set the horizontal time scale to 15 μ s.

- `scope.write(f'C1:VOLT_DIV {0.12} V')`

This will set the vertical voltage scale on channel 1 to 120 mV per division. Change the number after *C* to change the voltage scale on a different channel.

- `scope.write(f'TRIG_DELAY {-9e-5}')`

This will move the horizontal offset to -90 μ s.

- `scope.write(f'TRIG_MODE SINGLE')`

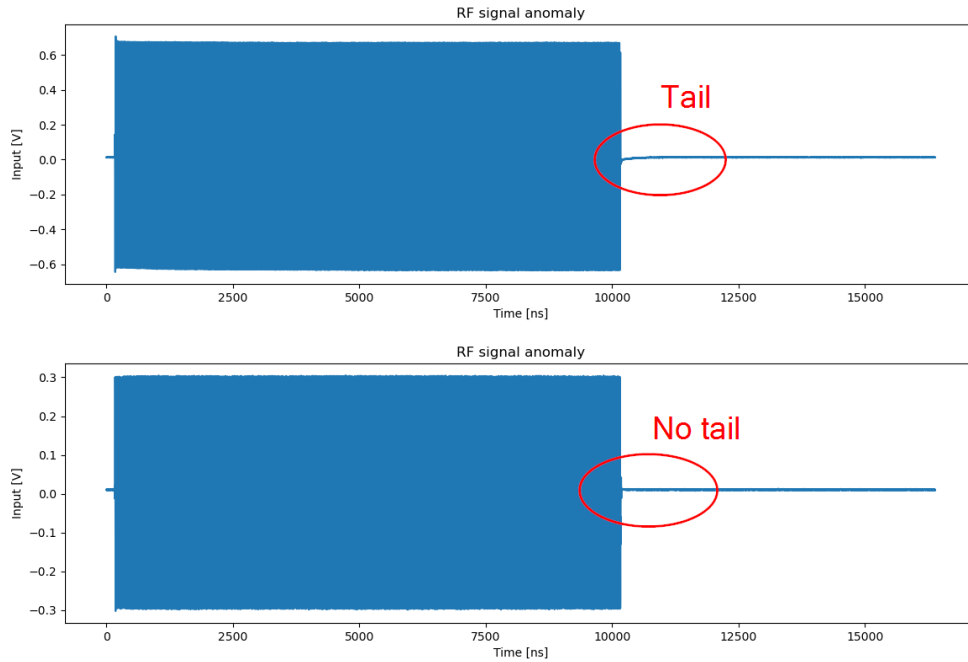
This will change the trigger mode to single. `SINGLE` can be replaced with `AUTO`, `NORMAL` or `STOP` to change to any 1 of the 4 modes.

5.2 Frequency mixer

The oscilloscope can only see signals up to about 2 GHz, so you can use a frequency mixer in order to play two high-frequency signals and mix them down to a low frequency to be seen on the oscilloscope.

We tried using two different frequency mixers, the Marki Microwave M20004LA and the Mini-Circuits ZEM-4300+. We found that the Marki Microwave tended to produce some kind

of 'tail' at the end of the RF pulses, as shown below, which the Mini-Circuits mixer did not, so I would recommend using the Mini-Circuits mixer.



5.3 Lock-in amplifier

The SRS lock-in amplifier can be used as a means to test long enough acquisitions, giving something very predictable to read in order to ensure that your acquisition is working as you expect. The frequency and amplitude of the signal can be adjusted with the *FREQ* and *AMPL* buttons and using the scroll wheel to adjust the values.

Then, the output can be connected to the QRM via a BCN cable with a BNC-F to SMA-M connector.

6 Updating Firmware and Drivers

Refer to the following compatibility matrix to check that your version of the firmware and qblox-instruments are compatible:

<https://pypi.org/project/qblox-instruments/>

If they are not compatible, the programs will likely not run.

6.1 Updating qblox-instruments

To install a specific version of qblox-instruments (for example, version 0.14.1) run the following in the python terminal:

```
pip install qblox-instruments==0.14.1
```

To install a different version, simply change the version number in the command.

6.2 Updating firmware

In order to check what firmware your QBlox Cluster is on, and to also get the cluster IP, simply run the following command in a Jupyter notebook:

```
!qblox-pnp list
```

This will output something like the following, showing the firmware version (0.9.1 in this case) and the cluster IP (192.168.137.2 in this case):

```
[2] # Run to get cluster IP
!qblox-pnp list

... Devices:
- 192.168.137.2: cluster_mm 0.9.1 with name "cluster-mm" and serial number 00015_2251_003
```

To install a different version of firmware:

- Go to

https://gitlab.com/qblox/releases/cluster_releases/-/releases

- Download the zip file of the release you desire to install.
- Ensure the QBlox Cluster is on and connected to the PC.
- Find the IP of the Cluster.
- In the terminal, run the following command:

```
qblox-cfg <cluster ip> update <filename.zip>
```

- For example, if you wish to use "cluster_releases-v0.9.1.zip" in the downloads folder, to update the cluster with IP 192.168.137.2:

```
qblox-cfg 192.168.137.2 update
C:\Users\BaughLaflamme\Downloads\cluster_releases-v0.9.1.zip
```

- The terminal will begin the update process, and ask if you would like to proceed [Y/N], type 'Y' and enter.
- The update should begin, the terminal will display when the update is done uploading.
- The cluster uses its LED's to indicate the state of the update.
 - red/purple indicates the updating of the CMM
 - yellow indicates the updating of the other modules
 - white appears when the updating is done

When attempting to update the firmware to version 0.9.1, I ran into an issue you may encounter. The terminal stated that the updates were finished, however the module LEDs were yellow, indicating that the update was still taking place. I left it like this for around 45 minutes, hoping it was just still updating and would be done soon. After that much time we decided to restart the cluster as we believed it became stuck in some state. Restarting allowed some code to run, however it was later stopped since the driver did not update to 0.9.1, and was still on 0.9.0. However, some of the stuff had updated, meaning some was old and some was new, and neither the old or new version of qblox-instruments was compatible. To remedy this, I simply attempted installing the firmware update again. This executed in less than a minute, and the cluster went back to its ready state. We are unsure of what cause it to get stuck, but just note that this may happen, and this is how we remedied it, in case you run into the same problem.

7 SPI Rack

To turn on the SPI Rack, there is a switch located at the back of the rack. In addition, the SPI Rack's DACs are powered by DC power supply source located on the right cabinet. Turn this power supply on, on both channels, to 5 V and 0.1 A to power the DACs.

7.1 Specifications

Parameter	Value
Range (V)	$\pm 4, \pm 2, +4$
Current (mA)	± 7.3
Impedance (Ω)	550
Resolution (bit)	18

7.2 Use with Python

The SPI rack can be connected via a Python notebook by importing the SpiRack class from qblox_instruments, using the line:

```
spi = SpiRack("SpiRack", "COM3")
```

Then the individual modules can be connected by using:

```
spi.add_spi_module(address, "D5a", "module1")
```

And a voltage can now be set to any value in volts using:

```
spi.module1.dac0.voltage(0.1)
```

The voltage range can be changed using:

```
spi.module1.dac0.span('range_4V_bi')
```

The argument `'range_4V_bi'` can also be switched with `'range_2V_bi'` or `'range_4V_uni'`. Other potentially useful commands are:

- `spi.temperature()` Returns the internal temperature of the SPI rack
- `spi.battery_voltages()` Returns the battery voltage
- `spi.set_dacs_zero()` Sets every connected DAC to 0 V

7.3 Use with Spanish Acquisition

The SPI Rack can also be controlled using Spanish Acquisition (SA) on the lab computer. Opening SA 3, the SPI rack can be connected by following these steps:

- Go to **Configuration > Devices...**
- Click **Add**
- Name your device with whatever name you'd like
- Double-click on **Disconnected** to open the connection menu
- Click **Load...** and select the SPI Rack .dev file
- Click **Connect**

If there is no .dev file, you can manually select a USB connection with the USB resource *COM3* with the implementation *Qblox* and *Qblox SPI Rack*.

Once the SPI Rack is connected, individual DAC voltages can be set by clicking on the SPI Rack connection again and going to the **resources** tab. By clicking on the name of the DAC you want, you can set the voltage under the **Value** tab.

To now perform a voltage sweep with one of the DACs, follow these steps:

- In the main SA menu, click **Add Output**
- A new row in the table should appear. You can give the variable you are sweeping a name
- Under **Resource**, enter the label of the DAC you are using. You can manually set the DAC label by double-clicking on the connection of the SPI Rack, going to the **Resources** tab and expanding on the DAC you wish to sweep.
- By double-clicking on the values list, you can input the start and stop voltages of your sweep as well as the number of steps to take in your sweep. Under **Quantity**, put the units for the sweep.
- Under the **Export** box, put the directory for wherever you'd like to save the data from the measurements taken during the sweep.
- By checking the box beside the variable name and clicking **Start**, the sweep will begin.

8 Slack Channel

If you are working with Qblox and do not have access to the Qblox Slack channel, you should ask to be added. It is an online messaging system that allows us to communicate with the Qblox engineers, and is a good place to ask questions and work our confusions. Some questions you have may have already been answered there.